

JavaScript

Boas práticas para
nomear variáveis

O que vamos ver hoje

01 Por que nomes importam

02 Clareza e intenção

03 CamelCase em JavaScript

04 Abreviações e nomes genéricos

05 Funções e booleanos

Ao final, você deve conseguir ler um trecho de código e melhorar os nomes sem mudar o comportamento.

01 Por que nomes importam?

Nomes são parte da comunicação do código. Eles ajudam pessoas a entenderem intenção, regra e contexto.

```
// funciona, mas explica pouco  
let x = 10;  
let y = 3;  
let z = x * y;
```

**O computador entende os dois.
O próximo dev entende melhor o segundo.**

```
// mesmo cálculo, mais claro  
let unitPrice = 10;  
let quantity = 3;  
let orderTotal = unitPrice *  
quantity;
```

Bons nomes reduzem comentários óbvios, bugs de interpretação e tempo perdido relendo código.

02 Regra 1: o nome revela a intenção

Evite nomes genéricos quando o contexto puder ser dito no próprio nome da variável.

pouco descritivo

```
let value = 10;  
let total = 45.90;  
let name = 'Ana';
```

Pergunta prática:
“Que informação essa variável guarda?”

mais claro

```
let noDeEstudantes = 10;  
let totalCarrinho = 45.90;  
let nomeEstudante = 'Ana';
```

Boa resposta:
substantivo + contexto
studentName, cartTotal, loginAttempts

03 Evite nomes genéricos

temp, aux e data parecem úteis, mas geralmente escondem a informação mais importante.

temp
↓
currentTemperature

data
↓
userProfile

aux
↓
formattedPrice

item
↓
selectedProduct

```
// antes  
let temp = 25;  
  
// depois  
let currentTemperature = 25;
```

04 Evite abreviações obscuras

Abreviar demais economiza segundos para escrever e cobra minutos para entender.

```
// difícil de ler  
let qtyStdnts = 10;  
let usrNm = 'Ana';  
let avgScr = 8.5;
```

```
// melhor  
let studentCount = 10;  
let userName = 'Ana';  
let averageScore = 8.5;
```

Pode abreviar?

- Sim, quando é uma sigla conhecida: id, URL, API, HTML.
- Evite abreviação inventada: qtdAln, usrNm, vlrTot.
- Prefira clareza quando estiver aprendendo.

16 Use camelCase em variáveis JavaScript

Convenção comum: primeira palavra minúscula e próximas palavras com inicial maiúscula.

```
// evite em variáveis JS  
let total_students = 10;  
let NumberOfStudents = 10;  
let numberofstudents = 10;
```

```
// padrão camelCase  
let totalStudents = 10;  
let numberOfStudents = 10;  
let averageScore = 8.5;
```

Como ler camelCase:

number + Of + Students → numberOfStudents

cart + Total → cartTotal

is + Approved → isApproved

06 Booleans devem soar como pergunta

Variáveis true/false ficam mais claras quando parecem responder sim ou não.

active → isActive

permission → hasPermission

edit → canEdit

save → shouldSave

```
let isApproved = nota >= 6;  
let hasAccess = role === 'admin';  
let canCheckout = cartTotal > 0;
```


07 Funções: verbo + complemento

Uma função faz alguma coisa. O nome deve dizer a ação e o objeto da ação.

```
// genérico demais
function doSomething() {
  // ...
}

function process() {
  // ...
}
```

```
// intenção explícita
function calculateAverageScore() {
  // ...
}

function formatCurrency() {
  // ...
}
```

Verbos comuns

calculate, get, set, create, update, delete, validate, format, render, show

Dica: se você precisa explicar “o que essa função faz”, talvez o nome ainda esteja fraco.

08 Listas e constantes também comunicam

Além de variáveis simples, arrays e constantes precisam revelar o tipo de informação que guardam.

```
// arrays no plural  
let students = ['Ana', 'Bia'];  
let grades = [8, 7.5, 9];  
let selectedProducts = [];
```

Arrays costumam ficar melhores no plural: students, products, grades.

```
// constantes em maiúsculas  
const MAX_LOGIN_ATTEMPTS = 3;  
const DEFAULT_LANGUAGE = 'pt-BR';
```

Constantes globais geralmente usam SNAKE_CASE em caixa alta.

Regra simples: o nome deve indicar se é um valor único, uma lista ou uma regra fixa.

Mini-refatoração guiada:

Sem mudar o comportamento, melhore os nomes abaixo.

Código original

```
let n = 'Ana';  
let p = 12.90;  
let q = 3;  
let t = p * q;  
console.log(n, t);
```

Perguntas para guiar

- Que pessoa ou entidade é n?
- p é preço de quê?
- q representa quantidade?
- t é total de qual operação?

Resposta possível:
customerName, unitPrice, quantity, orderTotal

Pratique:

Renomeie as variáveis para deixar o código mais legível.

- trabalhe em dupla por 3 minutos
- não altere os valores nem a lógica
- explique para a turma por que escolheu os nomes
- use camelCase

```
let a = 7.5;  
let b = 80;  
let c = a >= 6 && b >= 75;  
  
console.log(c);
```

**Pense em nomes como:
studentGrade, attendanceRate, isApproved**

12 Checklist rápido para nomear bem

Antes de seguir para o próximo trecho de código, revise mentalmente estes pontos.

✓ O nome revela a intenção?

✓ Evita genéricos como temp, aux e data?

✓ Usa camelCase em variáveis?

✓ Evita abreviação inventada?

✓ Booleano parece pergunta?

✓ Função começa com verbo?

```
// meta da aula  
let readableCode = true;  
let futureYouIsGrateful = true;
```

Quiz de revisão

Responda sem rodar o código. Depois conferimos juntos.

1. Qual nome é melhor: `value` ou `numberOfStudents`?

2. Por que `total_students` foge da convenção comum em JS?

3. Qual nome combina melhor com boolean: `active` ou `isActive`?

4. O que há de fraco em `function doSomething()`?

5. Dê um nome melhor para `let temp = 25` em um app de clima.

Bom código!

Nomes claros hoje evitam confusão amanhã.

Fonte-base: Medium / Lucas Pereira de Souza.